

Complete Machine Learning Course (Part 1)

These notes are compiled for personal learning purposes based on the course content.

CONTENTS

Module 1: Machine Learning Basics	2
1 AI vs ML vs DL	2
2 The Three Main Types of ML	3
3 Supervised Learning	6
4 Unsupervised Learning	6
5 Deep Learning	8
Module 2: Python Basics for Machine Learning	10
1 Python Basics	10
2 Python Control Flow	14
3 Python Functions	15
4 Python Numpy	17
Module 3: Important Libraries for Machine Learning	20
1 Python Pandas	20
2 Python Matplotlib	23
3 Python Seaborn	24
Module 4: Data Collection & Pre-processing	27
1 Collecting and Loading Datasets in Python	27
2 Importing Data Directly from Kaggle (Kaggle API)	28
3 Handling Missing Data	29
4 Data Standardization	31
5 Label Encoding	33
6 Train-Test Split	35
7 Handling Imbalanced Datasets	36
8 Feature Extraction from Text using TF-IDF	38
Module 5: Practical Machine Learning Examples	41
1 Sonar Rock vs Mine (Binary Classification)	41
2 Diabetes Prediction with SVM	44
3 Spam Email Classifier (Logistic Regression + TF-IDF)	46

MODULE 1: MACHINE LEARNING BASICS

1 AI vs ML vs DL

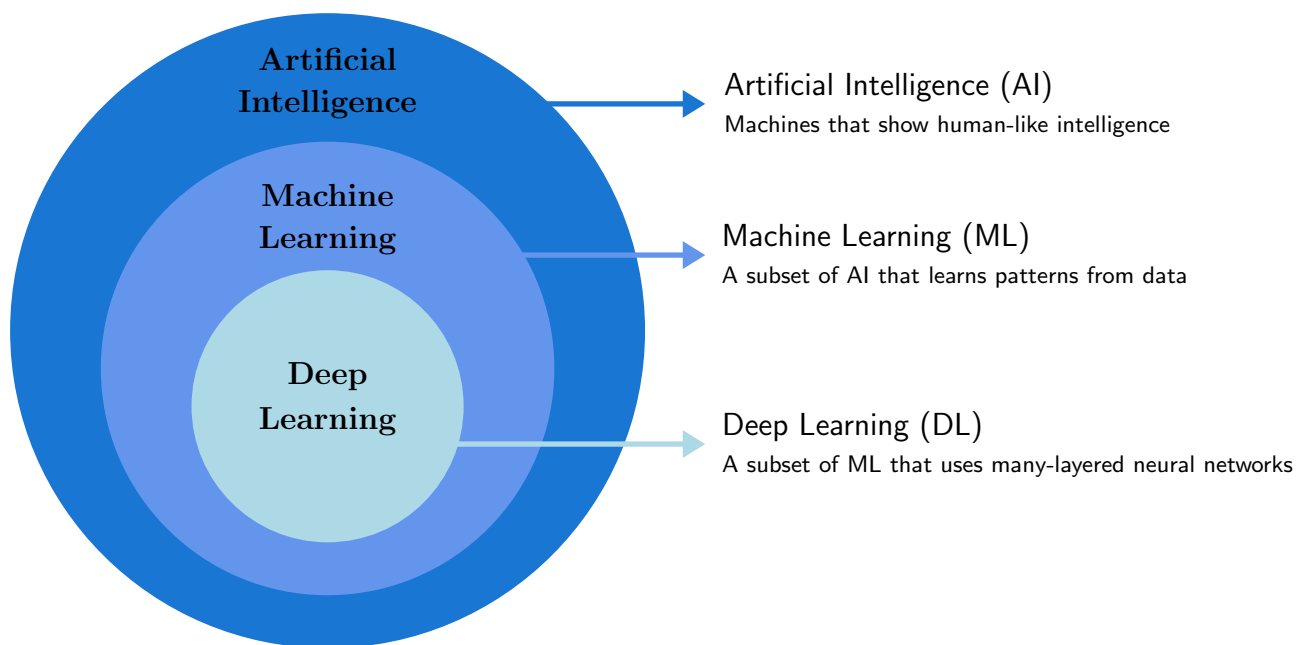


FIGURE 1. Hierarchy of Artificial Intelligence, Machine Learning, and Deep Learning.

(1) Artificial Intelligence (AI): The broad field concerned with creating systems that can perform tasks requiring human-like intelligence.

Intelligent vs. Non-Intelligent Machines

- **Non-intelligent machines:** Systems that operate strictly according to predefined rules or mechanical design, without the ability to adapt or learn from experience.

Examples: a bicycle, a basic washing machine, or a pocket calculator.

- **Intelligent systems:** Systems that can process data, recognize patterns, and adapt their behavior to new situations or inputs.

Examples: voice assistants such as Google Assistant or Siri (understand and respond to speech), autonomous vehicles (navigate and make driving decisions), and recommender systems on platforms like Netflix or YouTube (suggest content based on user behavior).

(2) Machine Learning (ML): A subfield of AI focused on algorithms that enable systems to automatically learn patterns from data and improve performance on a task without being explicitly programmed with task-specific rules.

Examples: The model is trained using many labeled images of *cats* and *dogs*. It learns what features make each animal different. Later, when it receives a new image, it predicts whether the image shows a cat or a dog.

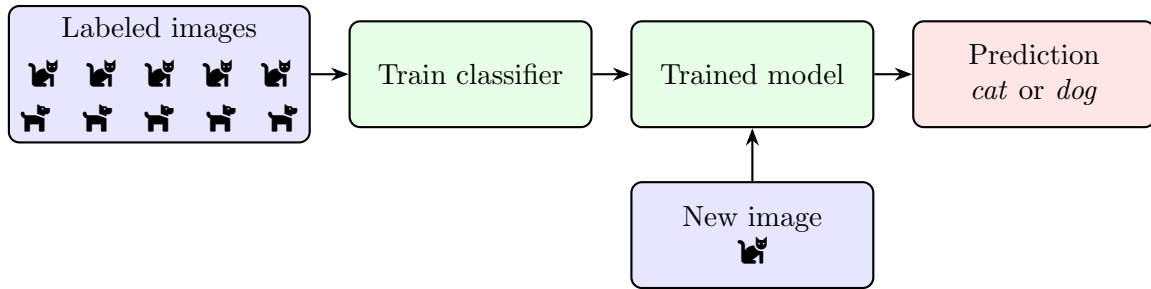


FIGURE 2. A model trained on labeled cat and dog images can classify new unseen images.

(3) Deep Learning (DL): A specialized branch of ML that uses multi-layered *artificial neural networks* (ANNs) to learn complex hierarchical representations from large amounts of data.

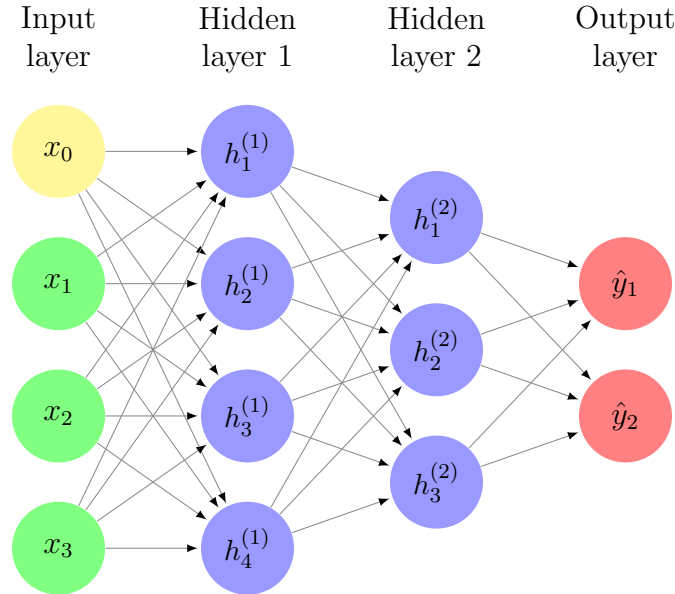


FIGURE 3. Illustration of a deep learning model: a neural network with two hidden layers.

2 The Three Main Types of ML

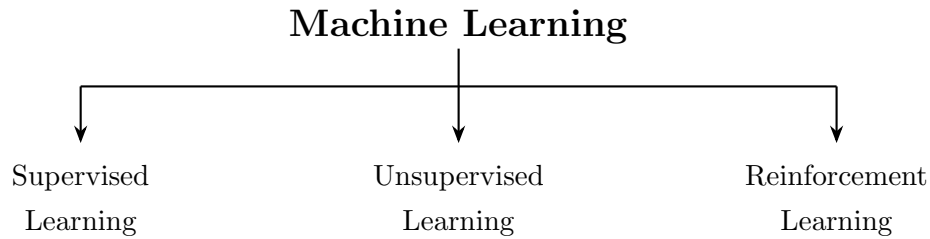


FIGURE 4. The three main types of machine learning: supervised, unsupervised, and reinforcement learning.

(1) Supervised Learning (with labels)

- **Idea:** The model is trained on examples where both the input x and the correct output y are known. It learns this relationship and can then predict y for new inputs.
- **Common tasks:**
 - **Classification:** predicting a category (for example, deciding whether an email is spam or not, or whether an image shows a cat or a dog).
 - **Regression:** predicting a number (for example, estimating the price of a house, the amount of rainfall, or a person's salary).

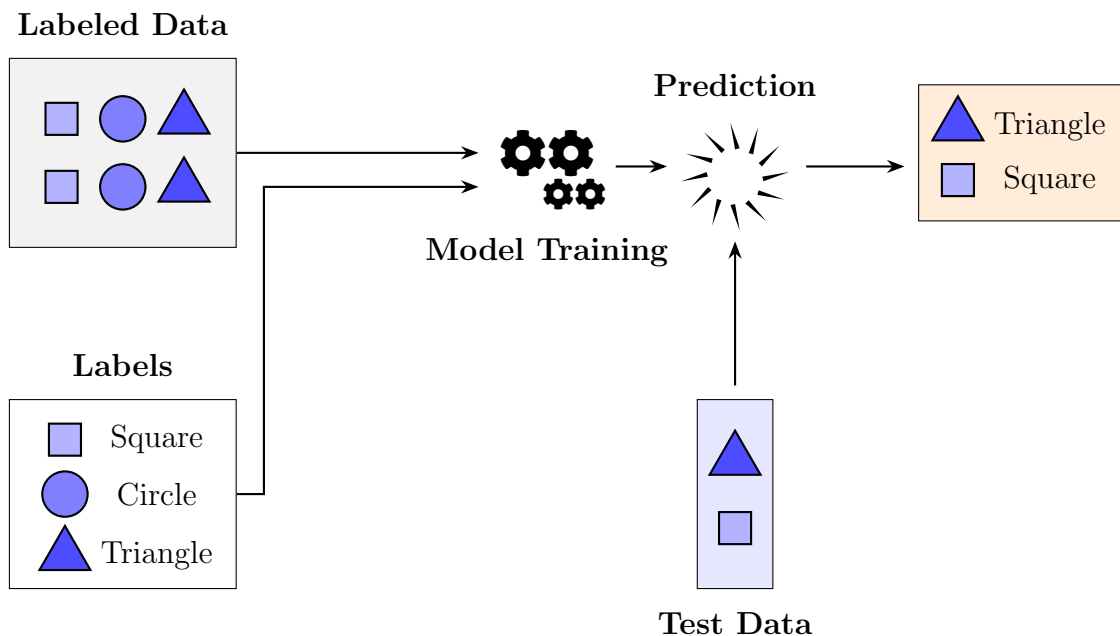


FIGURE 5. Supervised learning: training on labeled data to predict unseen inputs.

(2) Unsupervised Learning (without labels)

- **Idea:** Only the inputs are provided, without correct outputs. The model looks for patterns or structure in the data by itself.
- **Common tasks:**
 - **Clustering:** grouping similar items (for example, placing images of apples in one group and mangoes in another).
 - **Association:** finding items that often occur together (for example, “people who buy bread often buy milk”).
 - **Dimensionality reduction:** simplifying data by reducing the number of features while keeping the most important information (for example, PCA or autoencoders).

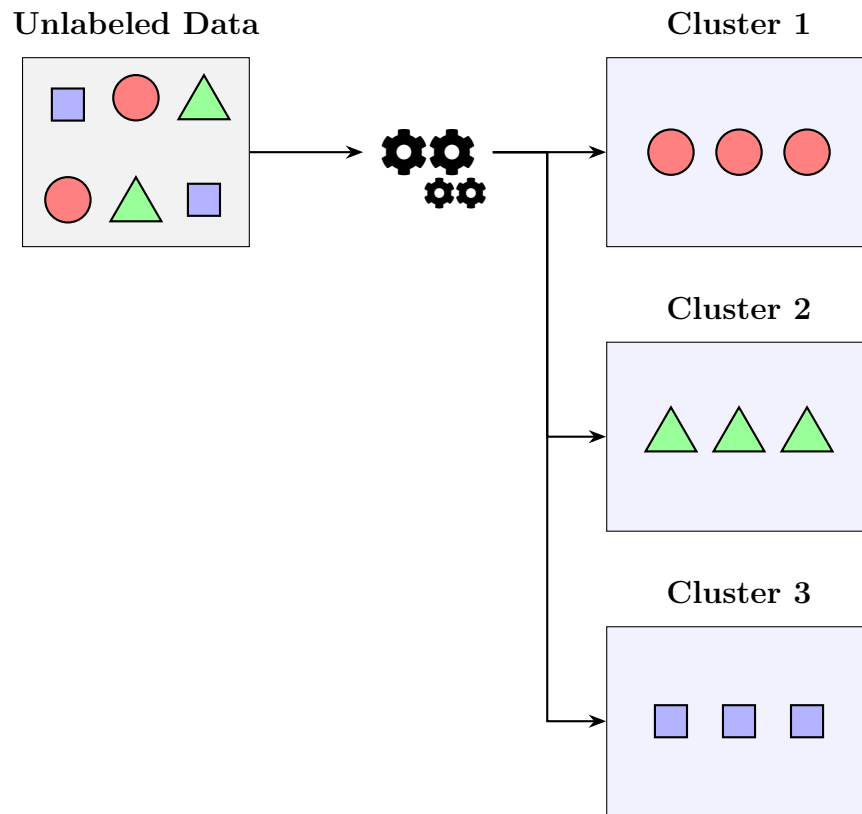


FIGURE 6. Unsupervised learning: clustering data into groups.

(3) Reinforcement Learning (RL)

- **Idea:** An **agent** learns by interacting with an **environment**. It takes actions, receives feedback in the form of **rewards**, and improves its decisions over time.

- **Example:** In chess or Go, the program tries different moves, receives feedback (winning, losing, or partial scores), and gradually learns a strategy that leads to better performance.

3 Supervised Learning

Supervised learning uses labeled datasets. The model is trained on input-output pairs, learning the mapping between data and labels.

Types of Supervised Learning

- (1) **Classification:** prediction of discrete classes (e.g., dog or cat, true or false).
- (2) **Regression:** prediction of continuous values (e.g., salary from years of experience, rainfall from temperature).

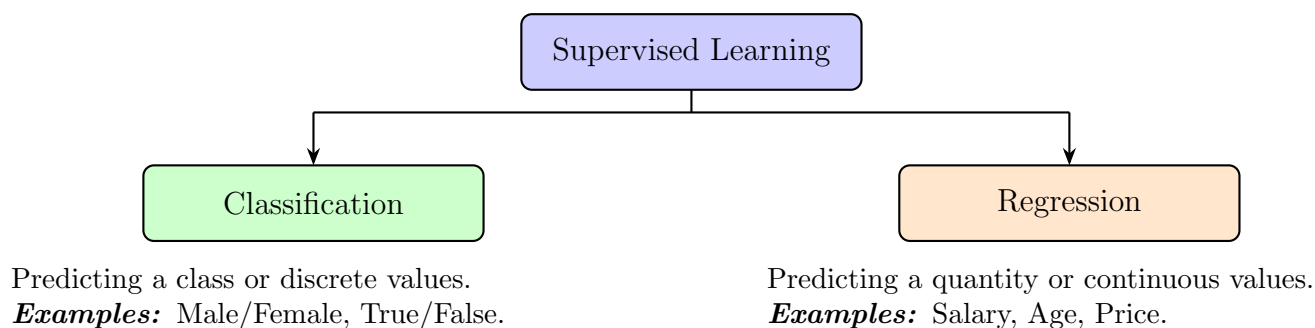


FIGURE 7. Types of supervised learning.

Common Algorithms

Classification	Regression
(1) Decision Trees	(1) Linear Regression
(2) Random Forests	(2) Polynomial Regression
(3) k-Nearest Neighbors	(3) Support Vector Regression

TABLE 1. Common Supervised Learning Algorithms

4 Unsupervised Learning

In unsupervised learning, algorithms work with unlabeled data and identify hidden structures or patterns without supervision.

Types of Unsupervised Learning

- (1) **Clustering:** grouping similar data points together.
- (2) **Association:** identifying relationships between data items.

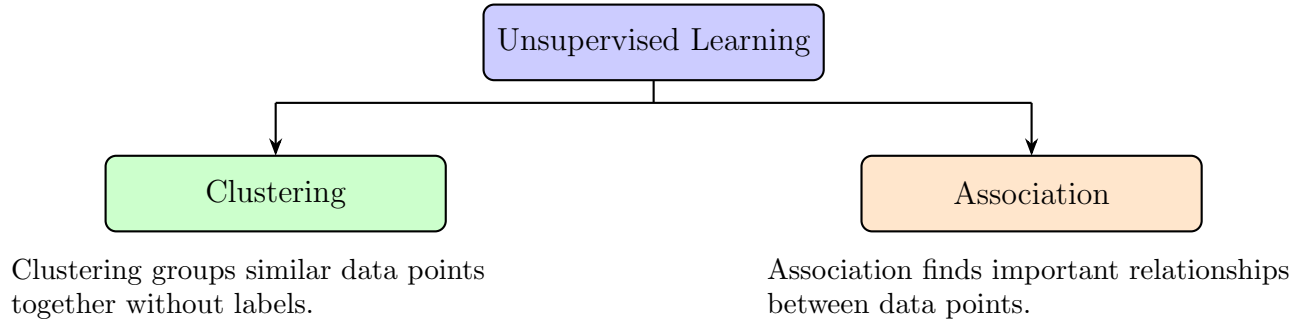


FIGURE 8. Types of unsupervised learning

Applications

Clustering

- Grouping news articles based on topics without prior labels.
- Segmenting mobile network users into groups based on call duration and internet usage.

Association

- Market basket analysis (e.g., customers who buy bread also tend to buy milk).
- Recommendation systems (e.g., suggesting superhero movies after watching Avengers).

Common Algorithms

Task	Algorithms
Clustering	(1) K-Means (2) Hierarchical Clustering
Dimensionality Reduction	(1) Principal Component Analysis (PCA)
Association	(1) Apriori (2) Eclat

TABLE 2. Common Unsupervised Learning Algorithms

5 Deep Learning

Deep learning is a subfield of machine learning that relies on **artificial neural networks (ANNs)**, which are inspired by biological neurons in the human brain.

- A biological neuron processes signals and passes them along its connections.
- Similarly, an artificial neuron applies a mathematical function and sends the result to the next layer.

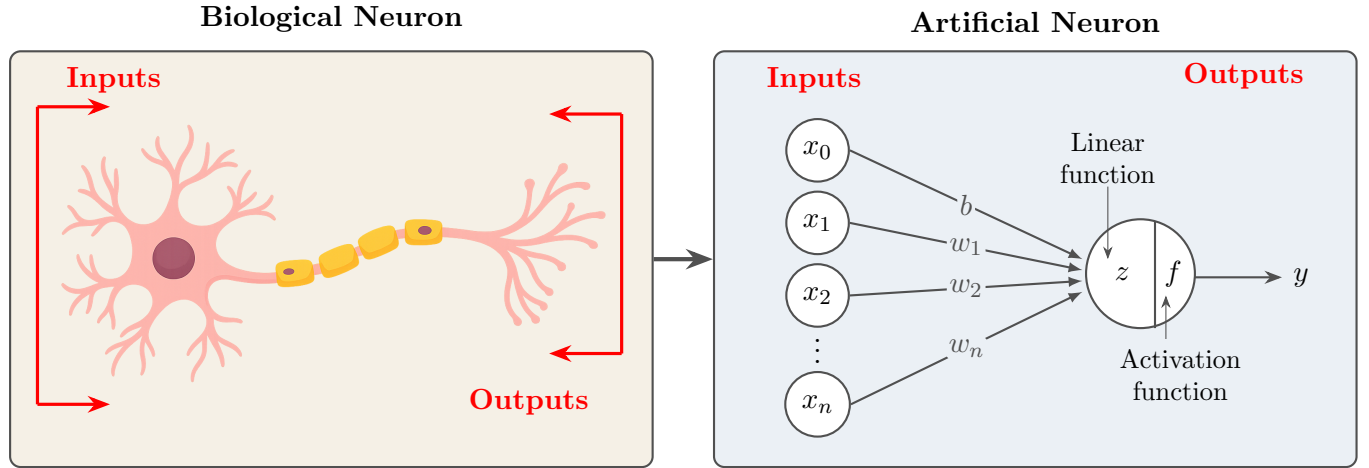


FIGURE 9. Comparison between a biological neuron and an artificial neuron.

Layers of Neural Networks

- (1) **Input Layer:** receives raw data (e.g., pixels of an image).
- (2) **Hidden Layers:** perform transformations and extract patterns.
- (3) **Output Layer:** produces the final prediction or classification.

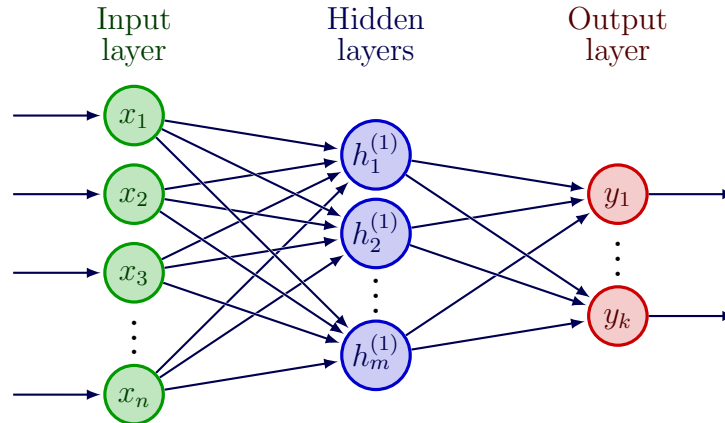


FIGURE 10. Neural network with input, hidden, and output layers.

Key Advantage of Deep Learning

The major difference between deep learning and traditional ML lies in **feature extraction**.

- **Machine Learning:** requires manual feature extraction (e.g., specifying wheels and shape for car detection).
- **Deep Learning:** automatically learns features from raw data, without manual intervention.

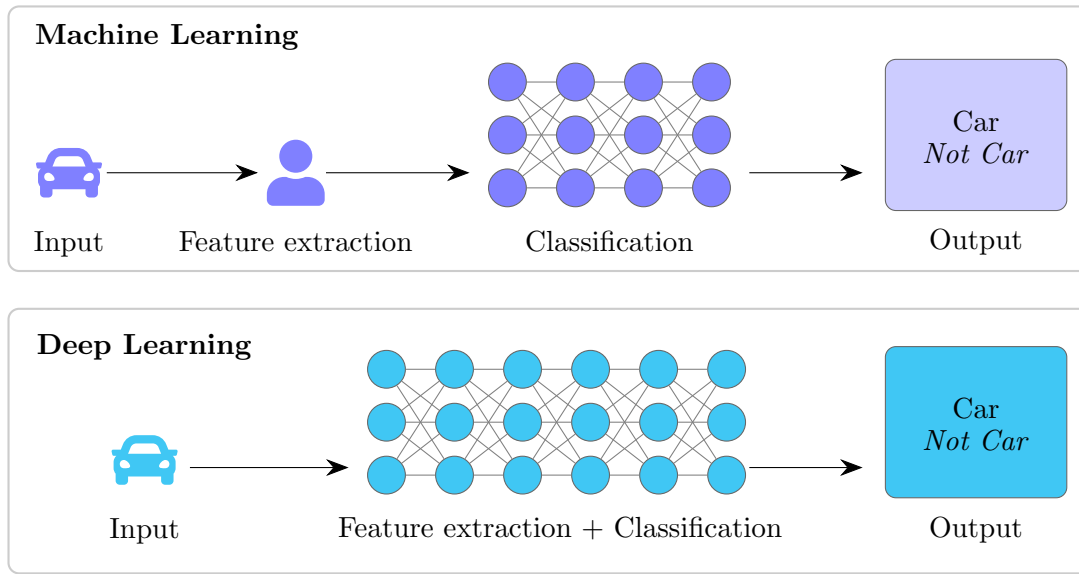


FIGURE 11. Contrast between classic machine learning (separate feature extraction and classification) and deep learning (end-to-end feature extraction and classification).

Applications of Deep Learning

- (1) **Healthcare**
 - Disease detection from medical scans (e.g., diabetic retinopathy).
 - Diagnosis support in hospitals.
- (2) **Autonomous Vehicles**
 - Self-driving cars use deep learning for object detection, navigation, and decision-making.
- (3) **Computer Vision**
 - Image recognition and face recognition systems in smartphones.
- (4) **Natural Language Processing (NLP)**
 - Chatbots, Google Assistant, Siri, Alexa.
 - Text understanding and translation.

MODULE 2: PYTHON BASICS FOR MACHINE LEARNING

Python is one of the most popular programming languages for machine learning and data science because it is:

- Readable and easy to learn.
- Rich in libraries (NumPy, pandas, scikit-learn, TensorFlow, PyTorch).
- A general-purpose language also used in web development, scripting, and automation.

1 Python Basics

Print Function

The `print()` function displays output. Strings must be enclosed in matching quotes.

```
print("Machine Learning")
print('Projects')
print("Machine" + " " + "Learning")    # Concatenation
print(8)                               # Numbers
print(8 + 3)                           # Arithmetic
```

Basic Data Types

Python supports five fundamental data types:

- **int**: integers (e.g., 10, -7)
- **float**: decimals (e.g., 3.14)
- **complex**: numbers with real and imaginary parts (e.g., 1+3j)
- **bool**: logical values (`True`, `False`)
- **str**: text (e.g., "hello")

```
print(type(8))           # <class 'int'>
print(type(5.3))         # <class 'float'>
print(type(1+3j))        # <class 'complex'>
print(type(True))        # <class 'bool'>
print(type("Hello"))     # <class 'str'>
```

Variables and Constants

- Variables act as containers and can change value.
- Constants are written in ALL_CAPS by convention, but Python does not enforce immutability.

```

superhero = "Iron Man"
print(superhero)          # Iron Man

superhero = "Thor"
print(superhero)          # Thor

PI = 3.14159
print(PI)                 # 3.14159
PI = 10
print(PI)                 # Reassignment allowed (not recommended)
                           # 10

```

Assignments

- Multiple variables can be assigned in one line.
- The same value can be assigned to many variables.

```

hero1, hero2, hero3 = "Iron Man", "Captain America", "Batman"
print(hero1, hero2, hero3)

x = y = z = 23
print(x, y, z)           # 23 23 23

```

Type Conversion

The `int()`, `float()`, and `str()` functions convert values between types.

```

x = 10
print(float(x))          # 10.0

y = 5.88
print(int(y))            # 5 (truncated, not rounded)

num1, num2 = "10", "20"
print(num1 + num2)       # "1020" (string concatenation)
print(int(num1) + int(num2)) # 30 (numeric addition)

```

Booleans

Booleans represent truth values and are often results of comparisons.

```

a, b = True, False
print(a, type(a))        # True <class 'bool'>
print(b, type(b))        # False <class 'bool'>

print(7 < 3)              # False
print(7 > 3)              # True

```

Strings

Strings support concatenation, repetition, and slicing.

```
print("Hello" * 3)           # HelloHelloHello
print("Machine" + " Learning") # Concatenation

s = "programming"
print(s[1:5])                # 'rogr'
print(s[0:10:2])             # 'pormrga' (every 2nd char)
```

Collections

Lists (mutable, ordered, allow duplicates)

- Enclose elements in [].
- Can store mixed data types.

```
lst = [2, 3, 1.8, "english", True]
lst.append(6)           # add element
print(lst[0], lst[2])   # indexing → 2, 1.8
print(len(lst))         # 6 elements
```

Tuples (immutable, ordered)

- Enclose elements in ().
- Support indexing but cannot be modified.

```
t = (2, 3, 4, 5)
print(t[0])           # 2
print(len(t))         # 4
```

Sets (unordered, unique elements)

- Enclose elements in { }.
- Do not allow duplicates and have no indexing.

```
s = {1, 2, 2, 3}
print(s)           # {1, 2, 3}
```

Dictionaries (key–value pairs)

- Enclose in { }, with key: value.

```
person = {"name": "Alice", "age": 25}
print(person["name"]) # Alice
```

Operators

Arithmetic Operators (+, -, *, /, **, %)

Perform mathematical calculations.

```
print(20 + 10)    # addition → 30
print(20 - 10)    # subtraction → 10
print(20 * 10)    # multiplication → 200
print(20 / 10)    # division → 2.0
print(20 ** 2)    # exponentiation → 400
print(20 % 3)     # modulus (remainder) → 2
```

Assignment Operators(=, +=, -=, *=, /=, **=, %=)

Assign values and update variables in place.

```
a = 5            # assign → 5
a += 3           # a = a + 3 → 8
a *= 2           # a = a * 2 → 16
```

Comparison Operators (==, !=, >, <, >=, <=)

Return True or False.

```
print(5 == 5)    # equal → True
print(5 != 3)    # not equal → True
print(7 > 3)     # greater than → True
print(7 <= 10)   # less than or equal → True
```

Logical Operators (and, or, not)

Combine multiple conditions.

```
x = 10
print((x > 5) and (x < 20))  # True and True → True
print((x > 15) or (x < 20))  # False or True → True
print(not (x > 5))           # not True → False
```

Identity Operators (is, is not)

Check if two variables refer to the same object in memory.

```
x, y = 5, 5
print(x is y)      # → True (same memory reference)
print(x is not y)  # → False
```

Membership Operators (in, not in)

Check if a value exists in a sequence.

```
nums = [1, 2, 3, 4]
print(3 in nums)      # → True
print(5 not in nums)  # → True
```

All of these operator examples are also included in the Jupyter notebook [PythonBasics.ipynb](#), where they can be executed and explored interactively.

2 Python Control Flow

If / Elif / Else

Two-way choice (if–else)

```
a, b = 15, 25    # two numbers to compare

if a > b:
    print("a is greater")
else:
    print("b is greater")    # runs because 25 > 15
```

Multi-way choice (if–elif–else)

```
a, b, c = 15, 25, 30    # three numbers

if a > b and a > c:
    print("a is greatest")
elif b > c:
    print("b is greatest")
else:
    print("c is greatest")    # runs because 30 is largest
```

Nested if (less common, but possible)

```
a, b, c = 20, 40, 60    # three numbers

if a > b:
    if a > c:
        print("a is greatest")
    else:
        print("c is greatest")
else:
    if b > c:
        print("b is greatest")
    else:
        print("c is greatest")    # runs because 60 is largest
```

Indentation note: After a colon :, the next lines must be indented. Those indented lines are part of the block.

For Loops

A **for** loop repeats a block of code a fixed number of times or goes through each element of a sequence.

Repeat a fixed number of times (using range)

```
for i in range(5):    # runs from 0 → 4
    print("Iteration:", i)
```

Iterate directly over a sequence

```
numbers = [50, 100, 150, 200]
for n in numbers:
    print("Value:", n)
```

While Loops

A **while** loop repeats as long as a condition remains **True**.

Run until condition becomes false

```
i = 0
while i < 5:          # loop continues while i < 5
    print(i)
    i += 1
```

Condition false at the start

```
i = 5
while i < 3:          # false immediately → loop skipped
    print(i)
    i += 1
```

Note:

- Use **for** when the number of iterations is known or when looping through a sequence (list, string, etc.).
- Use **while** when the repetition depends on a condition that changes during execution.

All of these operator examples are also included in the Jupyter notebook [PythonControlFlow.ipynb](#), where they can be executed and explored interactively.

3 Python Functions

A function is a named block of reusable code. It is defined with **def**, can take **parameters**, and may **return** values.

Basic definition & call

```
def greet(name):
    """Return a friendly greeting."""
    return f"Hello, {name}!"

print(greet("Ada"))          # Hello, Ada!
```

Function Parameters

- Positional and keyword arguments (mixable).
- Default values (make arguments optional).
- ***args** collects extra positional arguments.
- ****kwargs** collects extra keyword arguments.

```
def area(width, height=1):          # default parameter
    return width * height

def demo(*args, **kwargs):          # flexible
    return args, kwargs

print(area(5))                      # 5
print(area(5, 3))                   # 15
print(demo(1, 2, x=10))              # ((1, 2), {'x': 10})
```

Returning values (tuple unpacking)

```
def stats(nums):
    total = sum(nums)
    n = len(nums)
    return total, n, total / n

s, n, mean = stats([2, 4, 9])       # (15, 3, 5.0)
```

Docstrings (in-code help)

```
def normalize(text):
    """Return lowercase text without spaces at edges."""
    return text.strip().lower()

help(normalize)                     # shows the docstring above
```

Scope (variables inside/outside functions)

```
counter = 0

def tick():
    global counter          # rebinds global variable
    counter += 1

def outer():
    x = 0
    def inner():
        nonlocal x         # rebinds outer function variable
        x += 1
        return x
    return inner()
```

Worked example: factorial

```
def factorial(n):
    """Return n! for n >= 0. Raise error if negative."""
    if n < 0:
        raise ValueError("n must be >= 0")
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

print(factorial(5))        # 120
```

All of these operator examples are also included in the Jupyter notebook [PythonFunctions.ipynb](#), where they can be executed and explored interactively.

4 Python Numpy

What is NumPy?

NumPy is a Python library for *numerical computing*. It provides the `ndarray` object, which is similar to a Python list but with key advantages:

- Optimized for numerical operations.
- Supports vectorized (elementwise) math.
- Handles large datasets efficiently.

Why prefer arrays over lists?

- **Vectorized math:** write operations on whole arrays (no slow loops).
- **Performance:** arrays use contiguous memory + C speed under the hood.

Import NumPy

```
import numpy as np      # common shorthand
```

Creating arrays

```
np.array([1, 2, 3])      # from Python list
np.zeros((2, 3))         # 2x3 array of zeros
np.ones((2, 2))          # 2x2 array of ones
np.full((2, 3), 7)       # 2x3 array filled with 7
np.eye(3)                # 3x3 identity matrix
```

Ranges and random arrays

```
np.arange(0, 10, 2)       # [0 2 4 6 8]
np.linspace(0, 1, 5)      # 5 values evenly spaced 0 → 1
np.random.rand(2, 3)      # random floats in [0,1)
np.random.randint(1, 10, (2,3)) # random ints between 1 and 9
```

Random arrays (modern API)

```
rng = np.random.default_rng(42)    # recommended random generator
rng.random((2, 3))                 # random floats in [0, 1)
rng.integers(0, 100, (3, 3))       # random integers 0-99
```

Inspecting arrays

```
A = np.arange(12).reshape(3, 4)    # make 3x4 array
print(A)

A.ndim      # dimensions → 2
A.shape     # rows, cols → (3, 4)
A.size      # total elements → 12
A.dtype     # data type → int64 (on most systems)
```

Math on arrays (elementwise + broadcasting)

```
X = np.array([1, 2, 3])
Y = np.array([10, 20, 30])

X + Y        # [11 22 33] (elementwise addition)
X * Y        # [10 40 90] (elementwise multiplication)
X + 5        # [6 7 8] (scalar broadcast to all elements)
np.add(X, Y) # equivalent function call
```

Matrix operations

```
M = np.arange(6).reshape(2, 3) # 2x3
N = np.arange(9).reshape(3, 3) # 3x3

M @ N      # matrix multiplication
np.dot(M, N) # equivalent operation
```

Indexing and slicing

```
B = np.array([[5, 6, 7],
              [8, 9, 10],
              [1, 2, 3]])

B[0, 2]      # element at row 0,col 2 → 7
B[1:, :2]    # rows 1..end, first 2 columns
```

Reshape and transpose

```
C = np.arange(6).reshape(2, 3) # shape (2,3)
C.T      # transpose → shape (3,2)
C.reshape(3, 2) # reshape with same total size
```

Type conversion

```
E = np.array([1, 2, 3], dtype=np.int32)
E.astype(np.float64) # convert to float
```

Convert list or tuple to array

```
lst = [10, 20, 30]
arr = np.asarray(lst)
```

All of these operator examples are also included in the Jupyter notebook [PythonNumPy.ipynb](#), where they can be executed and explored interactively.

MODULE 3: IMPORTANT LIBRARIES FOR MACHINE LEARNING

1 Python Pandas

What is Pandas?

- **Pandas** is a Python library used for working with structured data such as tables, spreadsheets, and CSV files.
- It provides efficient tools for loading, exploring, cleaning, analyzing, and exporting data.
- The library is built around two main data structures:
 - **DataFrame**: a two-dimensional table containing rows and columns.
 - * Each column can store a specific type of data (numeric, string, or date).
 - * A DataFrame can be viewed as an entire spreadsheet or table.
 - **Series**: a one-dimensional labeled array representing a single column or row.
 - * Each Series consists of labels (called the **index**) and values.
 - * A Series can be viewed as a single column from a spreadsheet.

Setup and Import

```
import pandas as pd      # import pandas library and assign alias "pd"
import numpy as np       # import numpy library and assign alias "np"
```

Creating DataFrames

From a Python dictionary or list:

```
df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
print(df)
```

From a NumPy array:

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
df = pd.DataFrame(arr, columns=["C1", "C2", "C3"])
print(df)
```

From CSV or Excel files:

```
df = pd.read_csv("data.csv")          # read from CSV
df = pd.read_excel("data.xlsx", sheet_name=0) # read from Excel
```

From a sample dataset (example):

- **scikit-learn** (also called **sklearn**) is a popular Python library for *machine learning*.
 - It provides tools for training models and also includes small sample datasets for practice.
 - The example below shows how to load the **California Housing** dataset and convert it into a Pandas DataFrame.

```
from sklearn.datasets import fetch_california_housing

# Load the California Housing dataset
cal = fetch_california_housing(as_frame=True)

# Convert the data into a Pandas DataFrame
df = pd.DataFrame(cal.data, columns=cal.feature_names)

# Show the first few rows
df.head()

# Display the shape (rows, columns)
df.shape
```

Saving Data

```
# Save the DataFrame to a CSV file (commonly used format)
df.to_csv("output.csv", index=False)

# Save the DataFrame to an Excel file
df.to_excel("output.xlsx", index=False)
```

Inspecting Data

```
df.shape          # returns (number_of_rows, number_of_columns)
df.head()         # shows the first 5 rows
df.tail()         # shows the last 5 rows
df.info()         # column names, data types, and non-null counts
df.isnull().sum() # count of missing values in each column
```

Basic Statistics Pandas can calculate useful statistics easily:

```
df.count()        # non-null entries per column
df.mean(numeric_only=True) # mean
df.std(numeric_only=True)  # standard deviation
df.min(numeric_only=True)  # minimum
df.max(numeric_only=True)  # maximum
df.describe(include="all")  # summary table
```

Value Counts Used to count the number of occurrences of each unique value in a column.

```
# Count how many times each unique value appears in "Outcome"
df["Outcome"].value_counts()
```

Group By Used to divide the data into groups based on a specific column, then calculate statistics (such as mean, sum, or count) for each group.

```
# Group the data by the "Outcome" column and compute the average
df.groupby("Outcome").mean(numeric_only=True)
```

Selecting Rows and Columns

By position (using `iloc`)

```
row2 = df.iloc[1]          # second row
first_col = df.iloc[:, 0]  # first column
last_col = df.iloc[:, -1]  # last column
```

By label (using `loc`)

```
subset = df.loc[10:20, ["A", "B"]] # rows 10-20, columns A and B
```

Adding and Removing Columns or Rows

A new column can be added by assigning a list or array of values to it.

```
# Add a new column called "Price" with random values
df["Price"] = np.random.rand(len(df))

# Display the first few rows to confirm the addition
df.head()
```

Use the `drop()` function to remove columns or rows.

```
# Remove a column
df = df.drop(columns=["Price"])

# Remove specific rows (by index)
df = df.drop(index=[0, 3])
```

Correlation (Feature Relationships) Correlation measures how features change together. Values close to 1 mean strong positive relation, values near -1 mean strong negative relation.

```
corr = df.corr(numeric_only=True)
print(corr)
```

All of these operator examples, along with more detailed explanations, are included in the Jupyter notebook [PythonPandas.ipynb](#), where they can be executed and explored interactively.

2 Python Matplotlib

Matplotlib is Python's main plotting library. It helps display data as graphs, making patterns and relationships easier to see.

```
import matplotlib.pyplot as plt    # import plotting library
import numpy as np                 # import NumPy for data creation
```

Basic Line Plot (trend visualization):

A line plot illustrates the relationship between two variables. In the example below, the plotted line represents the mathematical function $y = x^2$.

```
x = np.arange(0, 10)    # create numbers from 0 to 9
y = x ** 2              # square each number

plt.plot(x, y)          # create a line plot
plt.title("y = x^2")    # add a title
plt.xlabel("x values")  # label for x-axis
plt.ylabel("y values")  # label for y-axis
plt.show()              # display the plot
```

Scatter Plot (relationships between variables):

A scatter plot visualizes the relationship between two numerical variables. Each point on the plot represents a single observation (row) from the dataset.

```
np.random.seed(1)
x = np.random.uniform(0, 10, 100)    # random x values
y = 2 * x + 3 + np.random.normal(0, 1, 100) # linear + noise

plt.scatter(x, y, color='purple', alpha=0.7)
plt.title('Scatter Plot: Linear Relationship')
plt.xlabel('x values')
plt.ylabel('y values')
plt.grid(True)
plt.show()
```

Bar Chart (categorical comparisons):

A bar chart is useful for comparing categorical values or counts. It can, for example, illustrate how many houses fall into different price ranges or show the frequency of outcomes within a dataset.


```
categories = ['Apples', 'Bananas', 'Cherries', 'Dates']
values = [120, 90, 150, 70]

plt.bar(categories, values, color='skyblue', edgecolor='black')
plt.title('Fruit Sales (units)')
plt.xlabel('Fruit Type')
plt.ylabel('Units Sold')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

Histogram (data distribution):

A histogram displays how frequently different values occur within a dataset. It is useful for identifying patterns, value ranges, and any skewness in the data distribution.

```
np.random.seed(2)
# generates 500 random height values with mean 170 cm and std 10 cm
heights = np.random.normal(170, 10, 500)

plt.hist(heights, bins=25, color='orange', edgecolor='black')
plt.title('Height Distribution of 500 People')
plt.xlabel('Height (cm)')
plt.ylabel('Count')
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.show()
```

Pie Chart (Proportion Visualization)

A pie chart displays proportions of categories in a dataset.

```
labels = ['Apples', 'Bananas', 'Cherries', 'Dates']
sizes = [30, 25, 25, 20]

plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=140)
plt.title("Fruit Sales Distribution")
plt.axis('equal') # make the pie a circle
plt.show()
```

All of these operator examples, along with more detailed explanations, are included in the Jupyter notebook [PythonMatplotlib.ipynb](#), where they can be executed and explored.

3 Python Seaborn

What is Seaborn?

Seaborn is a high-level data visualization library built on top of Matplotlib. It works naturally with **Pandas DataFrames**, making it ideal for exploring and visualizing real-world datasets.

Setup and Imports

```
import seaborn as sns          # main visualization library
import matplotlib.pyplot as plt # underlying plotting engine
import numpy as np             # for numerical data
import pandas as pd            # for data manipulation

# Apply Seaborn's default theme (grid, fonts, color palette)
sns.set_theme()
```

Built-in Datasets

Seaborn provides built-in datasets that can be loaded directly as Pandas DataFrames.

```
# Load sample datasets
tips = sns.load_dataset("tips")      # restaurant bills and tips
iris = sns.load_dataset("iris")      # flower measurements
titanic = sns.load_dataset("titanic") # passenger survival data

# Display a few rows
tips.head()
iris.head()
titanic.head()
titanic.shape # (rows, columns)
```

Relational Plot (Faceting and Encodings)

Visualize relationships between two numeric variables, while using color, style, or size to show additional information.

```
sns.relplot(
    data=tips,          # dataset to visualize (Pandas DataFrame)
    kind="scatter",     # type of relational plot ('scatter' or 'line')
    x="total_bill",     # x-axis variable: total amount of the bill
    y="tip",            # y-axis variable: amount of the tip
    col="time",         # separate plots for 'Lunch' and 'Dinner'
    hue="smoker",       # color represents whether customer is a smoker
    style="smoker",     # marker shape also represents smoker status
    size="size",        # point size shows party size (number of people)
    sizes=(20, 200),    # smallest and largest marker sizes
    height=4,           # height (inches) of each subplot
    aspect=1.1         # width-to-height ratio
)
plt.show()
```

Scatter Plots

Used to compare continuous features and identify clusters or relationships.

```
# Sepal length vs Petal length
sns.scatterplot(data=iris, x="sepal_length", y="petal_length", hue="species")
plt.title("Iris: Sepal Length vs Petal Length")
plt.show()

# Sepal length vs Petal width
sns.scatterplot(data=iris, x="sepal_length", y="petal_width", hue="species")
plt.title("Iris: Sepal Length vs Petal Width")
plt.show()
```

Count Plots (Categorical Counts)

A count plot shows how many observations fall into each category.

```
# Passenger class distribution
sns.countplot(data=titanic, x="class")
plt.title("Passenger Count by Ticket Class")
plt.show()

# Survival counts
sns.countplot(data=titanic, x="survived")
plt.title("Titanic: Survival Counts (0 = No, 1 = Yes)")
plt.show()
```

Bar Plot with Hue (Comparing Subgroups)

A bar plot shows the average value of a variable per category. For binary outcomes (like survived), the bar height represents the proportion of positive cases.

```
sns.barplot(
    data=titanic,
    x="sex", y="survived", hue="class",
)
plt.title("Survival Rate by Sex, Split by Class")
plt.ylabel("Proportion Survived")
plt.show()
```

All of these operator examples, along with more detailed explanations, are included in the Jupyter notebook [PythonSeaborn.ipynb](#), where they can be executed and explored.

MODULE 4: DATA COLLECTION & PRE-PROCESSING

1 Collecting and Loading Datasets in Python

Why Data Matters?

Machine learning models identify patterns from data to make predictions or classifications. For example:

- In image tasks (e.g., cats vs. dogs), models learn features such as edges and shapes.
- In healthcare, predictive models use patient data (lab results, scans) to assess disease risk.

Large and diverse datasets lead to stronger, more reliable models. High-quality, well-labeled data is essential for accurate learning and fair predictions.

Common Dataset Sources

- **Kaggle** ([kaggle.com](https://www.kaggle.com)): A large online platform that hosts thousands of datasets. It also offers community notebooks, tutorials, and data science competitions.
- **UCI Machine Learning Repository** (archive.ics.uci.edu): A long-standing academic resource that provides classic datasets such as *Iris*, *Wine Quality*, and *Breast Cancer*.
- **Google Dataset Search** (datasetsearch.research.google.com): A search engine designed to help locate datasets across multiple websites and organizations.
- **Other reliable sources**: Government portals (data.gov, data.gov.uk), open research repositories, and company data portals often share public datasets for analysis and education.

Example A: Finding and Downloading a Dataset (Google Dataset Search → Kaggle)

- (1) Open [Google Dataset Search](#) and search for `Boston house price`.
- (2) Select a Kaggle link (competition or dataset page) and sign in if required.
- (3) On the Kaggle **Data** tab, click **Download**. The dataset is typically provided as a `.zip` file containing one or more `.csv` files.

Example B: Download from UCI Machine Learning Repository (Iris Dataset)

- (1) Search for *UCI iris dataset* and open the official dataset page.
- (2) Click **Data Folder** and download `iris.data`.
- (3) Rename the file to `iris.csv` to ensure tools recognize it as a CSV file.

- (4) Inspect the file in a text editor to confirm structure (comma-separated values; last column is the species name).

Uploading and Loading Data in Python

Datasets can be stored locally on a computer or downloaded from the internet. Once the file (for example, a CSV file) is available, it can be loaded easily using the Pandas library.

```
import pandas as pd

# Load a CSV file (replace with your filename)
dataset = pd.read_csv("housing.csv")

# Display first 5 rows and shape
dataset.head()
dataset.shape    # (number of rows, number of columns)
```

2 Importing Data Directly from Kaggle (Kaggle API)

The Kaggle API allows datasets to be downloaded directly into Python, making it easy to access data for analysis or modeling.

Step 1: Install and Verify Kaggle Package

```
# Install the Kaggle package if not already installed
!pip install kaggle

# Check installation (should show version info)
!kaggle --version
```

Step 2: Configure Kaggle API Credentials

- (1) Go to your Kaggle profile → **Account** → click **Create New API Token**.
 - This downloads a file called `kaggle.json`.
- (2) Move this file into a hidden folder called `.kaggle` inside your home directory.

```
import os

# Create the Kaggle directory if it doesn't exist
os.makedirs(os.path.expanduser("~/.kaggle"), exist_ok=True)

# Move kaggle.json to the correct location
os.rename("kaggle.json", os.path.expanduser("~/.kaggle/kaggle.json"))

# Set the correct permissions for security
os.chmod(os.path.expanduser("~/.kaggle/kaggle.json"), 0o600)
```

Step 3: Download a Dataset from Kaggle

Example: The “House Prices” competition dataset.

```
import pathlib

# Create a folder to store the dataset
pathlib.Path("data").mkdir(exist_ok=True)

# Download the dataset zip file
!kaggle competitions download -c house-prices-advanced-regression-techniques
  -p data/

# Unzip the contents into the "data" folder
!unzip -o data/house-prices-advanced-regression-techniques.zip -d data/
```

Step 4: Load the Data into Pandas

```
import pandas as pd

# Read the training and testing data
train = pd.read_csv("data/train.csv")
test = pd.read_csv("data/test.csv")

# Preview data
train.head()
```

Downloading a Public Dataset

```
# Example: Wine Reviews dataset
!kaggle datasets download -d zynicide/wine-reviews -p data/
!unzip -o data/wine-reviews.zip -d data/

df = pd.read_csv("data/winemag-data-130k-v2.csv")
df.head()
```

7. Quick Data Quality Checks Before using data in a model, run a few simple diagnostics:

```
dataset.info()           # column names, data types, and non-null counts
dataset.isnull().sum()   # count of missing values in each column
dataset.describe()       # statistical summary for numeric columns
```

3 Handling Missing Data

Why This Matters

In most datasets, some cells are missing (displayed as NaN in Python). Machine learning algorithms require complete data to work correctly.

Main Strategies

There are two general approaches to dealing with missing values.

- **Imputation:** Replace missing values with estimated ones computed from existing data.
 - The **mean** is suitable when the data distribution is approximately symmetric.
 - The **median** is preferred for skewed distributions or data with outliers.
 - The **mode** is appropriate for categorical or nominal features.
- **Dropping:** Remove rows or columns containing missing values when the proportion of missing data is small or when a variable is largely incomplete and contributes little information.

1. Detecting Missing Values

The first step is to determine the extent and location of missing data in the dataset.

```
import pandas as pd

# Load dataset
df = pd.read_csv("data.csv")

# Count missing values in each column
df.isnull().sum().sort_values(ascending=False)
```

2. Visualizing Missing Data and Feature Distributions

Visualization assists in deciding which type of imputation is most appropriate for a given feature.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Select a numeric feature to analyze
num_col = "feature_name"

# Visualize feature distribution
sns.histplot(df[num_col], kde=True)
plt.title(f"Distribution of {num_col}")
plt.xlabel(num_col.capitalize())
plt.ylabel("Frequency")
plt.show()
```

- If the distribution is symmetric, the mean and median are similar and either can be used for imputation.
- When the distribution is skewed or contains extreme values, the median provides a more robust estimate.

3. Imputation Techniques

(a) Median Imputation (skewed or outlier-prone data)

```
median_value = df["feature_name"].median()
df["feature_name"].fillna(median_value, inplace=True)
```

(b) Mean Imputation (symmetric numeric data)

```
mean_value = df["feature_name"].mean()
df["feature_name"].fillna(mean_value, inplace=True)
```

(c) Mode Imputation (categorical variables)

```
for col in ["feature_A", "feature_B", "feature_C"]:
    mode_value = df[col].mode(dropna=True)[0]
    df[col].fillna(mode_value, inplace=True)
```

4. Dropping Missing Data

Dropping rows or columns removes missing entries entirely. Dropping should be used with caution, as it may reduce the dataset size and eliminate potentially informative records.

```
# Drop rows with any missing values
df_clean = df.dropna(how="any")

# Check new dataset size
df_clean.shape
```

The Jupyter notebook [DataPreprocessing.ipynb](#) provides executable examples that demonstrate the detection, visualization, and treatment of missing values.

4 Data Standardization

Why Standardize?

- In real-world datasets, numerical features often have very different ranges and units.
 - For example, one column might represent **Age** (0–100), while another represents **Income** (in thousands).
- Such scale differences can cause certain features to dominate others, making it difficult for many algorithms (especially those that rely on distance or gradient calculations) to perform well.

Standardization is a preprocessing technique that rescales numerical features so they share a similar scale.

Instead of keeping raw values in their original units, each feature is transformed into a standardized form using the formula:

$$\tilde{x} = \frac{x - \mu}{\sigma} \quad (1)$$

where μ is the feature mean and σ is the feature standard deviation.

After standardization:

- Each feature has approximately mean 0 and standard deviation 1.
- Optimization-based algorithms converge faster and more stably.
- Distance-based models (e.g., KNN, SVM) treat all variables with equal importance.

Step 1: Import required libraries

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")
```

Step 2: Load the Dataset

Any numerical dataset can be used for this demonstration. The example assumes the data file contains several numeric columns and one target variable.

```
# Load dataset (replace with any dataset containing numeric features)
df = pd.read_csv("your_dataset.csv")

# Preview first few rows
df.head()
```

Step 3: Separate Features and Target Column

Before standardizing, separate the input features (X) from the target (y). Only numeric features should be standardized.

```
# Separate features and target
X = df.drop(columns=["target_column"]).values
y = df["target_column"].values
```

Step 4: Split Data into Training and Test Sets

Training data is used to learn the standardization parameters (mean and standard deviation). The same transformation is then applied to the test data.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,          # 20% of data for testing
    random_state=42,        # reproducibility
    stratify=y              # keep class ratio same if classification task
)

print("Training shape:", X_train.shape)
print("Testing shape :", X_test.shape)
```

Step 5: Fit and Apply Standardization

The `StandardScaler` computes the mean and standard deviation from the training data and applies scaling to both training and test sets.

```
# Initialize the scaler
scaler = StandardScaler()

# Learn parameters (mean, std) from the training data
scaler.fit(X_train)

# Apply the same transformation to training and test data
X_train_std = scaler.transform(X_train)
X_test_std  = scaler.transform(X_test)
```

Step 6: Verify the Transformation

After scaling, the training data should have mean ≈ 0 and standard deviation ≈ 1 . This confirms the standardization process is correct.

```
# Calculate mean and standard deviation of scaled features
train_means = X_train_std.mean(axis=0)
train_stds  = X_train_std.std(axis=0)
```

The Jupyter notebook [DataStandardization.ipynb](#) provides an interactive walkthrough of these steps with visualizations and hands-on experiments.

5 Label Encoding

- Most machine learning models work only with numeric values and cannot process text labels directly.
 - The `LabelEncoder` from `scikit-learn` is used to convert each unique text label into a corresponding integer value.
 - Each category is assigned a number based on alphabetical order by default.
 - For example: `apple` $\rightarrow$ 0, `banana` $\rightarrow$ 1, `cherry` $\rightarrow$ 2.

Dependencies

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
```

Example 1: Breast cancer dataset (binary labels)

Goal: encode column diagnosis (B/M) into target (0/1).

```
# (1) Load CSV -> DataFrame
cancer_data = pd.read_csv("data.csv")

# (2) Peek
cancer_data.head()

# (3) Class counts before encoding
# B = benign
# M = malignant
cancer_data["diagnosis"].value_counts()

# (4) Fit encoder on the text labels and transform to integers
le_bc = LabelEncoder()
cancer_data["target"] = le_bc.fit_transform(cancer_data["diagnosis"])

# (5) Confirm counts
cancer_data["target"].value_counts()
```

Example 2: Iris dataset (three classes)

Goal: encode column species into target (0/1/2).

```
# (1) Load CSV -> DataFrame
iris_data = pd.read_csv("iris.csv")

# (2) Peek
iris_data.head()

# (3) Class counts before encoding
iris_data["species"].value_counts()

# (4) Fit encoder and create numeric target
le_iris = LabelEncoder()
iris_data["target"] = le_iris.fit_transform(iris_data["species"])

# (5) Confirm counts
iris_data["target"].value_counts()
```

The Jupyter notebook [DataEncoding.ipynb](#) provides an interactive walkthrough of these steps with visualizations and hands-on experiments.

6 Train-Test Split

Machine learning follows a logical sequence of steps that transform raw data into an evaluated model. Each step plays a distinct role in building reliable and accurate predictions.

Machine Learning Process:

- (1) **Collect Data:** Gather raw data from files, APIs, or databases.
- (2) **Preprocess Data:** Handle missing values, standardize numerics, encode categoricals.
- (3) **Analyze Data:** Explore patterns and trends using statistics and visualizations.
- (4) **Split Data:** Divide the dataset into training and testing sets.
- (5) **Train Model:** Fit a learning algorithm using the training data.
- (6) **Evaluate Model:** Assess how well the model performs on unseen data.

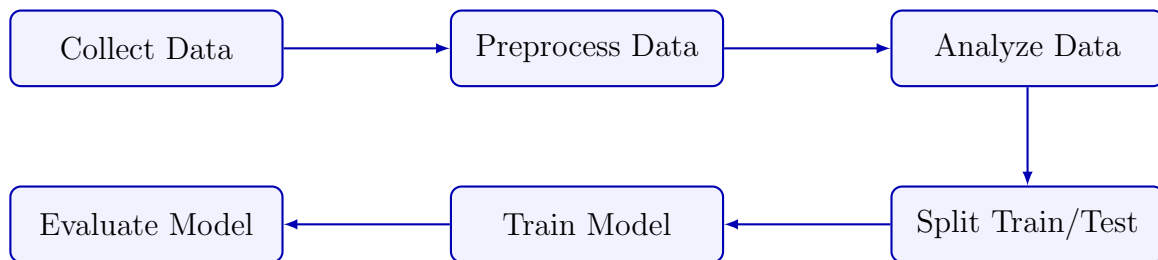


FIGURE 12. Overview of a typical machine learning workflow.

Purpose of the Split

A model's goal is to learn general patterns, not memorize the data. To measure how well it generalizes, a portion of the dataset is held back for evaluation.

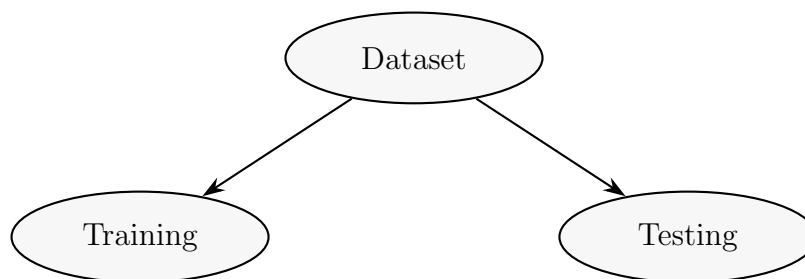


FIGURE 13. Dataset split into training and testing subsets.

- Typically, **80–90%** of the data is used for training.
- The remaining **10–20%** is reserved for testing.
- Evaluation must always be done on the test set, never on the same data used for training.

Example: Diabetes Dataset (diabetes.csv)

The following example demonstrates how to perform a proper train–test split and standardize the data.

```
# 1. Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 2. Load dataset (e.g., Pima Indians Diabetes dataset)
df = pd.read_csv("diabetes.csv")

# 3. Separate features (X) and target (y)
X = df.drop(columns=["Outcome"]) # feature columns
y = df["Outcome"]               # target: 0 = non-diabetic, 1 = diabetic

# 4. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,          # 20% of data for testing
    random_state=2,          # reproducible random split
    stratify=y               # keep class ratios balanced
)

print(X.shape)              # shape of full dataset
print(X_train.shape)        # shape of training data
print(X_test.shape)         # shape of test data

# 5. Standardize numeric features (fit only on training data)
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)
```

The Jupyter notebook [DataSplitting.ipynb](#) provides an interactive walkthrough of these steps with visualizations and hands-on experiments.

7 Handling Imbalanced Datasets

What is an imbalanced dataset?

An imbalanced dataset occurs when one class has far more examples than the other.

- **Example:** In a credit card fraud dataset, there may be **284,000 normal transactions** but only **492 fraudulent ones**.
 - A model trained on such data might predict “normal” almost every time, achieving about **99.8% accuracy**.
 - * However, this high accuracy is misleading because the model misses nearly all fraud cases.

- * For such problems, metrics like **precision**, **recall**, and **F1-score** provide a better picture of true performance.

Why this is a problem

- When the majority of data points belong to a single class, the model tends to favor that class in order to minimize overall errors.
 - As a result, it often fails to detect the rare but important cases, such as instances of fraud, disease, or product defects.

Objective

The goal is to make the training data more balanced so the model can learn both classes properly.

Load the dataset

```
import numpy as np
import pandas as pd

# Load the credit card fraud dataset
df = pd.read_csv("your_dataset.csv")

# Preview the first few rows
print(df.head())

# Check class distribution (e.g., 0 = majority class, 1 = minority class)
print(df["Class"].value_counts())
```

Separate classes

```
# Split into majority and minority classes
majority = df[df["Class"] == 0]
minority = df[df["Class"] == 1]
print(majority.shape, minority.shape)
```

Balance classes by **undersampling** the majority

```
# Match number of majority samples to number of minority samples
n_minority = len(minority)

# Randomly select a subset of majority class examples
majority_sample = majority.sample(n=n_minority, random_state=42)

# Combine and shuffle the two balanced groups
balanced = (
    pd.concat([majority_sample, minority], axis=0)
    .sample(frac=1, random_state=42)
    .reset_index(drop=True)
)
```

The Jupyter notebook [DataImbalanceHandling.ipynb](#) provides an interactive walkthrough of these steps with visualizations and hands-on experiments.

8 Feature Extraction from Text using TF–IDF

- Machine learning algorithms require numerical data, but most real-world information comes in text form.
 - To make text usable for models, it must be converted into numbers that represent meaning and importance.
 - * This process is known as **text feature extraction**.

Common Approaches

- **Bag of Words (BoW)**: Counts how many times each word appears in a document, ignoring word order and context.
- **TF–IDF (Term Frequency–Inverse Document Frequency)**: Improves BoW by giving higher weights to words that are unique or rare, and lower weights to common words such as *the* or *is*.

Mathematical Formulation

$$\text{TF}(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in } d}, \quad (2)$$

$$\text{IDF}(t) = \log\left(\frac{N}{1 + \text{df}(t)}\right), \quad (3)$$

$$\text{TF} \cdot \text{IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t) \quad (4)$$

Where:

- t is a term or word
- d is a document, such as a sentence, article, or review
- N is the total number of documents in the dataset
- $\text{df}(t)$ is the number of documents that contain the term t

The final TF–IDF score shows how useful a word is for identifying the content of a document compared to the entire collection.

Step 1 Import Required Libraries

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
```

Step 2 Load a Text Dataset

A small example dataset is used here to demonstrate the concept. Any dataset containing text fields, such as product reviews, news headlines, or tweets, can be analyzed in the same way.

```
# Load a simple text dataset
data = {
    "text": [
        "NASA launches a new satellite into orbit",
        "The baseball game was postponed due to rain",
        "Astronauts train for long space missions",
        "The team won the championship after extra innings"
    ]
}

# Convert the data into a DataFrame
df = pd.DataFrame(data)

# Display the dataset
print(df)
```

Step 3 Create and Configure a TF-IDF Vectorizer

The `TfidfVectorizer` converts text into numerical feature vectors that represent the TF-IDF weight of each term.

```
# Initialize the TF-IDF vectorizer
tfidf = TfidfVectorizer(
    lowercase=True,          # convert all text to lowercase
    stop_words="english",    # remove common English words
    ngram_range=(1, 2),     # include single words and two-word phrases
    sublinear_tf=True       # apply logarithmic scaling to term frequency
)

# Transform the dataset into TF-IDF features
X_tfidf = tfidf.fit_transform(df["text"])

# Display shape of the TF-IDF matrix
print("TF-IDF matrix shape:", X_tfidf.shape)
```

Step 4 Inspect the Vocabulary and TF-IDF Values

Each column in the matrix corresponds to a term. The vectorizer stores these terms in alphabetical order.

```
# Display the first 10 learned terms
print("First 10 terms in vocabulary:")
print(tfidf.get_feature_names_out()[:10])
```


Step 5 Identify the Most Important Words in a Document

By examining the highest TF-IDF scores in a single document, it is possible to see which words best describe its content.

```
# Select the first document
doc_index = 0

# Get feature names and TF-IDF scores
feature_array = tfidf.get_feature_names_out()
tfidf_scores = X_tfidf[doc_index].toarray().flatten()

# Find top 5 most important words
top_indices = tfidf_scores.argsort()[::-1][:5]
top_words = feature_array[top_indices]

print("\nTop 5 words for document 0:")
print(top_words)
```

Interpretation

- For example, the sentence about satellites and space will have top words such as *NASA*, *satellite*, or *orbit*, while the sentence about baseball will highlight *team*, *game*, or *innings*.
- TF-IDF automatically focuses on words that capture the meaning of each text sample.

The Jupyter notebook [DataTextVectorization.ipynb](#) provides an interactive walkthrough of these steps with visualizations and hands-on experiments.

MODULE 5: PRACTICAL MACHINE LEARNING EXAMPLES

1 Sonar Rock vs Mine (Binary Classification)

- Develop a **Logistic Regression** model to classify sonar signals as either **Rock (R)** or **Mine (M)** using the well-known UCI Sonar dataset.
- Each record contains **60 numerical features** representing signal intensity readings from different frequencies, and one **target label** indicating whether the signal was reflected from a rock or a metal mine.

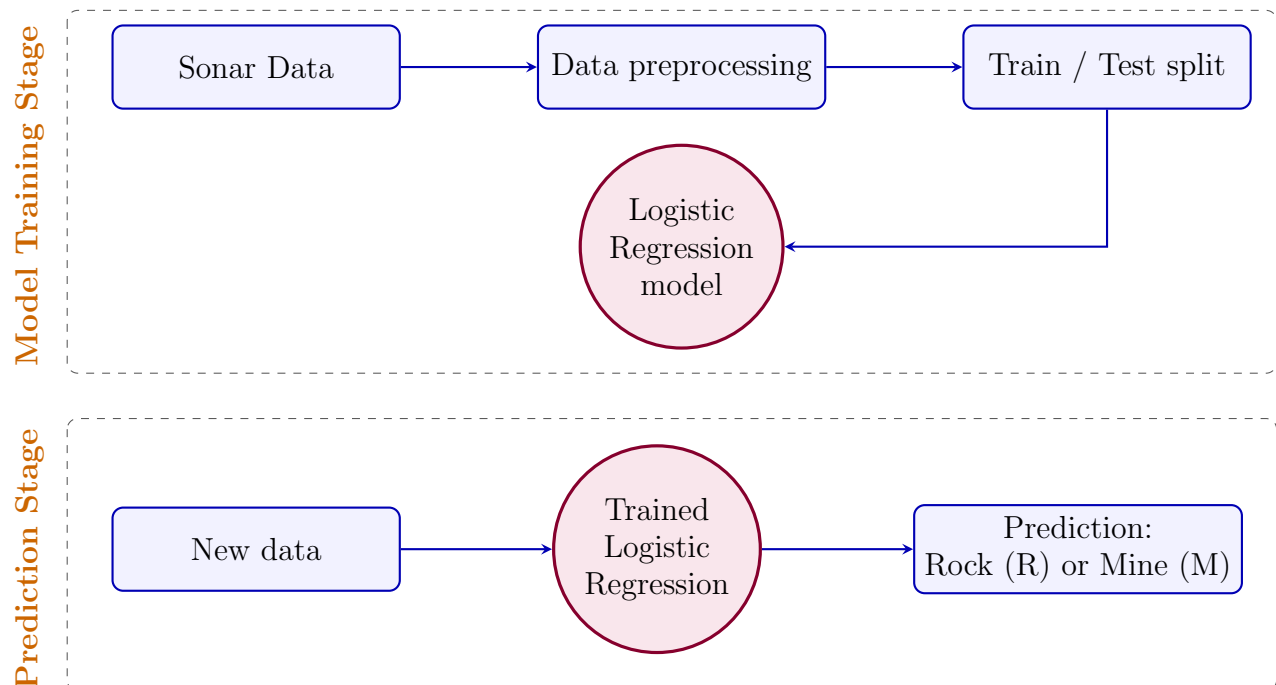


FIGURE 14. Logistic regression workflow.

(1) Import required libraries

```

# Core libraries
import numpy as np
import pandas as pd

# Scikit-learn tools
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
  
```

(2) Load the dataset

```
# Load CSV file (no header row in this dataset)
# Each row = sonar signal, last column = label ("R" or "M")
sonar = pd.read_csv("sonar.csv", header=None)

# View shape and first few rows
print("Dataset shape:", sonar.shape)    # (208, 61) → 60 features + 1 label
print(sonar.head())                    # display first 5 rows
```

Output:

```
Dataset shape: (208, 61)
   0      1      2      3      4      5  ...  56      57      58      59      60
0  0.0200  0.0371  0.0428  0.0207  0.0954  0.0986  ...  0.0167  0.0180  0.0084  0.0090  R
1  0.0453  0.0523  0.0843  0.0689  0.1183  0.2583  ...  0.0191  0.0140  0.0049  0.0052  R
2  0.0262  0.0582  0.1099  0.1083  0.0974  0.2280  ...  0.0244  0.0316  0.0164  0.0095  R
3  0.0100  0.0171  0.0623  0.0205  0.0205  0.0368  ...  0.0073  0.0050  0.0044  0.0040  R
4  0.0762  0.0666  0.0481  0.0394  0.0590  0.0649  ...  0.0015  0.0072  0.0048  0.0107  R
```

Explanation: The dataset has 208 observations. Each observation has 60 numeric attributes (signal readings) and a label column: "R" = Rock, "M" = Mine.

(3) Split features and labels

```
# Separate independent (X) and dependent (y) variables
X = sonar.iloc[:, :-1].astype(float) # first 60 columns = numeric features
y = sonar.iloc[:, -1]                # last column = labels ("R" or "M")

print("Feature matrix shape:", X.shape)
print("Label vector shape:", y.shape)
```

Output:

```
Feature matrix shape: (208, 60)
Label vector shape: (208,)
```

Explanation: X contains all signal readings, y contains the corresponding class for each reading.

(4) Split into training and test sets

```
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.10,      # 10% of data used for testing, 90% for training
    stratify=y,          # keeps Rock/Mine ratio the same in both sets
    random_state=1       # ensures reproducible random split
)

print("Train size:", X_train.shape)
print("Test size:", X_test.shape)
```

Output:

Train size: (187, 60)
 Test size: (21, 60)

Explanation: The Sonar dataset contains 208 total samples. `train_test_split()` assigns 90% (187 samples) to the training set and 10% (21 samples) to the test set. Using `stratify=y` ensures that both subsets maintain the same proportion of Rock (R) and Mine (M) labels, avoiding class imbalance.

(5) Train the Logistic Regression model

```
# Initialize model (increase max_iter to avoid convergence warnings)
model = LogisticRegression(max_iter=1000)

# Fit (train) the model using the training data
model.fit(X_train, y_train)

print("Model training complete.")
```

(6) Evaluate the model

```
# Make predictions for training and test data
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

# Evaluate accuracy
print("Training accuracy:", accuracy_score(y_train, train_pred))
print("Testing accuracy :", accuracy_score(y_test, test_pred))
```

Output:

Training accuracy: 0.8342245989304813
 Testing accuracy : 0.7619047619047619

(7) Predict a single example

```
def predict_one(feature_list):
    """
    Predict whether the sonar reading represents a Rock or a Mine.
    feature_list: list of 60 numeric values.
    """
    # Convert the list to a NumPy array and reshape to (1, 60)
    arr = np.asarray(feature_list, dtype=float).reshape(1, -1)

    # Use the trained model to predict ("R" or "M")
    label = model.predict(arr)[0]

    # Return a readable label
    return "Rock" if label == 'R' else "Mine"
```

```
# Example: take one sonar reading from the test set
example = X_test.iloc[0].tolist()
print("Predicted:", predict_one(example))
```

This example is implemented in [Example_1_Rock_vs_Mine_Classification.ipynb](#).

2 Diabetes Prediction with SVM

- Develop a **Support Vector Machine (SVM)** model to classify whether a patient is **diabetic (1)** or **non-diabetic (0)** using the Pima Indians Diabetes dataset.

Workflow

- (1) Load `diabetes.csv` into a pandas DataFrame.
- (2) Split into features **X** and label *y* (**Outcome**).
- (3) Train/test split with `stratify=y`.
- (4) Fit `StandardScaler` on training data only. Transform train and test.
- (5) Train a linear SVM: `SVC(kernel="linear")`.
- (6) Evaluate accuracy on train and test.
- (7) Build a small predictor for a single patient row.

(1) Import required libraries

```
# Core libraries
import numpy as np
import pandas as pd

# Scikit-learn tools
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
```

(2) Load the dataset

```
# Load CSV file (each row = patient data, last column = Outcome)
df = pd.read_csv("diabetes.csv")

# View shape and first few rows
print("Dataset shape:", df.shape)
print(df.head())
```

Explanation: The dataset contains 768 samples and 9 columns. Each row represents a patient's medical data. The last column, `Outcome`, is the label: 1 = diabetic, 0 = non-diabetic.

(3) Split features and labels

```
# Separate input features (X) and target label (y)
X = df.drop(columns=['Outcome']).astype(float) # 8 numeric features
y = df['Outcome'].astype(int)                 # 0 or 1

print("Feature matrix shape:", X.shape)
print("Label vector shape:", y.shape)
```

Explanation: X contains the medical measurements, while y holds the binary diagnosis (0 or 1).

(4) Split into training and test sets

```
# 80% for training, 20% for testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,      # 20% test data
    stratify=y,          # maintain class ratio
    random_state=2       # reproducible split
)

print("Train size:", X_train.shape)
print("Test size:", X_test.shape)
```

Explanation: Out of 768 samples, 614 are used for training and 154 for testing. The `stratify=y` parameter ensures both sets have the same proportion of diabetic and non-diabetic cases.

(5) Standardize the data

```
# Scale features so each has mean = 0 and std = 1
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train) # fit on training data
X_test_std = scaler.transform(X_test)      # apply same scale to test
```

Explanation: Standardization improves performance for models like SVM by ensuring all features contribute equally to the decision boundary.

(6) Train the SVM model

```
# Create and train a linear Support Vector Machine
model = SVC(kernel='linear', random_state=2)
model.fit(X_train_std, y_train)
print("Model training complete.")
```

Explanation: The Support Vector Machine (SVM) learns a decision boundary that best separates diabetic and non-diabetic cases in the standardized feature space. Using a **linear** kernel means the model tries to find a straight-line (or hyperplane) separation between the two classes.

(7) Evaluate the model

```
# Make predictions on train and test sets
train_pred = model.predict(X_train_std)
test_pred  = model.predict(X_test_std)

# Calculate accuracy
print("Training accuracy:", accuracy_score(y_train, train_pred))
print("Testing accuracy :", accuracy_score(y_test, test_pred))
```

(8) Predict a single example

```
# Define the correct feature order (must match training data columns)
FEATURE_ORDER = [
    "Pregnancies", "Glucose", "BloodPressure", "SkinThickness",
    "Insulin", "BMI", "DiabetesPedigreeFunction", "Age"
]

def predict_one(row_in_feature_order):
    """
    Predict diabetes for a single patient record.
    Input: list of 8 numeric values matching FEATURE_ORDER.
    """

    # Convert input to a DataFrame with feature names
    input_df = pd.DataFrame([row_in_feature_order], columns=FEATURE_ORDER)

    # Standardize using the same scaler fitted on the training data
    input_std = scaler.transform(input_df)

    # Predict using the trained SVM model
    prediction = model.predict(input_std)[0]

    # Convert numeric output (0 or 1) to readable text
    return "Diabetic" if prediction == 1 else "Non-diabetic"

# Example: test the function with one patient record from the test set
example = X_test.iloc[0].to_list()
print("Prediction:", predict_one(example))
```

Explanation: The function converts a new patient's data into the correct numeric format, applies the same scaling, and returns a readable prediction.

This example is implemented in [Example_2_Diabetes_SVM.ipynb](#).

3 Spam Email Classifier (Logistic Regression + TF-IDF)

- Build a **Logistic Regression** model that classifies messages as **Spam (0)** or **Ham (1)** using TF-IDF features extracted from email text.

Workflow

- (1) Load the dataset (`mail_data.csv`) containing email messages and labels.
- (2) Clean and encode labels (`spam` → 0, `ham` → 1).
- (3) Split into training and test sets using `stratify=y`.
- (4) Convert text data into TF-IDF feature vectors.
- (5) Train a Logistic Regression model.
- (6) Evaluate model accuracy on both training and test data.
- (7) Build a small function to classify new messages.

(1) Import required libraries

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

(2) Load and inspect the dataset

```
# Load dataset (two columns: Category and Message)
df = pd.read_csv("mail.csv")

# Handle alternate column names (v1/v2)
if set(["v1", "v2"]).issubset(df.columns):
    df = df.rename(columns={"v1": "Category", "v2": "Message"})

# Display shape and first few rows
print("Dataset shape:", df.shape)
print(df.head())
```

Explanation: The dataset contains text messages labeled as either `ham` (not spam) or `spam`. Each row includes one message and its corresponding label.

(3) Encode labels and clean data

```
# Replace missing values with empty strings
df = df.fillna("")

# Convert labels to lowercase and encode
df["Category"] = df["Category"].str.lower()
df["Label"] = df["Category"].map({"spam": 0, "ham": 1})
```

Explanation: Here, `spam` = 0 and `ham` = 1. This makes the labels numeric so they can be used in a machine learning model.

(4) Split into features and labels

```
X = df["Message"].astype(str)
y = df["Label"].astype(int)

# 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=3
)

print("Train size:", X_train.shape)
print("Test size:", X_test.shape)
```

Explanation: The data is split so that both training and test sets contain a similar ratio of spam and ham messages.

(5) Convert text into TF-IDF features

```
# Initialize TF-IDF vectorizer (fit only on training data)
tfidf = TfidfVectorizer(min_df=1, stop_words="english", lowercase=True)
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print("TF-IDF feature shape (train):", X_train_tfidf.shape)
```

Explanation: TF-IDF converts raw text into numeric feature vectors that reflect how important each word is within the dataset.

(6) Train the Logistic Regression model

```
# Create and train Logistic Regression model
model = LogisticRegression(max_iter=1000)
model.fit(X_train_tfidf, y_train)
print("Model training complete.")
```

Explanation: Logistic Regression is trained on the TF-IDF features to distinguish spam from ham messages. It learns word patterns commonly found in spam emails.

(7) Evaluate model performance

```
# Predict on both training and test sets
train_pred = model.predict(X_train_tfidf)
test_pred = model.predict(X_test_tfidf)

# Compute accuracy
print("Training accuracy:", accuracy_score(y_train, train_pred))
print("Testing accuracy :", accuracy_score(y_test, test_pred))
```

Explanation: High test accuracy indicates the model generalizes well and can accurately detect spam in unseen data.

(8) Predict new messages

```
def classify_email(text):  
    """Classify a new email as Spam or Ham."""  
    features = tfidf.transform([text])  
    pred = model.predict(features)[0]  
    return "Ham (not spam)" if pred == 1 else "Spam"  
  
# Example predictions  
print(classify_email("Please review the attached document."))  
print(classify_email("You won a FREE iPhone! Click now!"))
```

Explanation: This helper function uses the trained TF-IDF model and Logistic Regression classifier to label new emails as either spam or non-spam in real time.

This example is implemented in [Example_3_Spam_Email_Classifier.ipynb](#).